DevDoc Studio — SRS status report

Where the application stands against the 52 requirements in the Software Requirements Specification, what remains, and a recommended order to build it in. Each requirement was checked against the current code; the companion spreadsheet *SRS-DevdocStudio-Gap-Analysis.xlsx* carries the same status and notes per row.

52 REQUIREMENTS ASSESSED

24 DONE

11 PARTIALLY DONE

17 NOT STARTED

AREA	PROGRESS	DONE	PARTIAL	NOT STARTED	TOTAL	% DONE
Document editor & UI		11	0	0	11	**100%**
Test manager		11	6	4	21	**52%**
Release management		0	4	10	14	**0%**
Non-functional		2	1	3	6	**33%**
All requirements		**24**	**11**	**17**	**52**	**46%**

Key points

1. **Document editing is complete (11 of 11 done):** permissions, dashboard, editor, VS Code export and download all work, enforced by the backend. Data management is now admin-only, with a full server backup and a confirmed reset; the session audit log is recorded by the server (007, 008, 011).
2. **Test management is on the backend and nearly complete.** Suites, cases, runs and milestones moved from the browser to the server (Redis) and are shared by every user. Projects and sub-sections, revision history, run comparison, consolidated reports and 5,000-case suites in ~0.3 s are done; what remains is the execution engine. Correction: 016 (runs from selected cases or sections) was reported done on 2 Nov, but only whole-suite runs work - it is now Partial and back in Phase 0.
3. **Remote execution still runs against a mock Test PC.** The real hardware API is not available yet, so the server registry, backend worker and live updates will be built against a test-server API we define, with the mock as its first implementation.
4. **Release management is the largest gap (0 of 14 done)** and is being built now (Phase 3): releases, RC builds, gates, sign-offs, promotion and an immutable audit log.
5. **Recently completed:** all app data moved off the browser - documents are server-only (Postgres, cached in Redis), with version history and the Doc Bin on the server; Phases 0 and 1 are done; navigation measured at under 0.4 s per screen (071).

Recommended order of work

Every requirement from the original plan appears exactly once below; green ones are done since 2 Nov. The order follows dependencies: fix what is broken and cheap first, then build the backend foundation that test execution and release management both rely on. Effort is relative: S = days, M = 1–2 weeks, L = several weeks.

PHASE	FOCUS	REQUIREMENTS (COLOUR = CURRENT STATUS)	WHY IN THIS ORDER
0	**Quick wins & fixes** Effort: S · Done except 016	008 011 007 013 018 015 024 016	Done, apart from 016, reopened on 4 Nov: Add Test Run's 'specific cases' and 'dynamic filtering' options need implementing, plus a per-section choice (a small fix). Delivered: admin-only data management with a server backup and reset, a server-recorded session audit log, complete test case fields (priorities renamed to Critical / High / Medium / Low), issue IDs on failed results, CSV/JSON/XML import-export, and CSV/HTML/PDF run reports.
1	**Move test management to the backend** Effort: L · Done	012 014 073 031 025	Done. Test suites, cases and runs now live on the server (Redis), shared, searchable and auditable: Projects and sub-sections, revision history, the 5,000-case performance target, consolidated reports and run-to-run comparison.
2	**Real test execution engine** Effort: L · Next	019 075 020 021 022 023 072 028 026 027	Replace the mock Test PC: a server registry with encrypted credentials, a backend polling worker (arq on the existing Redis) with configurable intervals and full job states, live push updates to the UI, metrics, then scheduled runs and Jenkins/Bitbucket triggers. The hardware API is not available, so it will be built against a test-server API we define.
3	**Release management** Effort: L · In progress	037 038 039 040 035 034 042 043 044 045 041 033 046 036	In progress - started before Phase 2, which waits on the Test PC API. Built in dependency order: create releases and milestones, register RC builds and their artifacts, link test runs to RCs, evaluate gates from real run results and blockers, collect multi-role sign-offs, then promotion to Final, overrides and an immutable audit log.
4	**Security & platform hardening** Effort: M · Partly done	074 076 071 030	Navigation measured under 0.4 s per screen (071 done). HTTPS/TLS 1.3 waits for a deployment target, SSO/OAuth2 sign-in for an identity provider, and connecting existing repositories for a Git host.

Chip colours: done partial not started . Phase 1 is done, so Phases 2 and 3 can run in parallel if two people are available; Phase 3 is being built first.

Document Editor & UI

DDS-SRS-001 – 011 · 11 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
1	DDS-SRS-001	**List in-progress documents; open in editor**	Done	Documents list shows in-progress (Draft / In Review) documents; select to open in the editor. Enforced server-side too (GET /api/documents).
2	DDS-SRS-002	**Users edit only their own documents**	Done	Non-admins can only edit/delete documents they authored (permissions.ts; re-checked by the API: 403 otherwise).
3	DDS-SRS-003	**Admin views all documents**	Done	Admins see every document, any status, and can open it in the editor.
4	DDS-SRS-004	**Admin edits all documents**	Done	Admins can edit and delete all documents (frontend + API).
5	DDS-SRS-005	**Dashboard landing page with document stats**	Done	Dashboard is the landing page after login: total / approved / in-review / draft counts and latest Google Drive upload status.
6	DDS-SRS-006	**Sidebar entry for the document editor**	Done	Sidebar item 'Document Editor'.
7	DDS-SRS-007	**Sidebar: doc editor, data management, code editor**	Done	Sidebar has Document Editor, Coder (code editor) and, for admins, a Data Management entry that opens System & Storage.
8	DDS-SRS-008	**Data management restricted to admins**	Done	Data Management is admin-only in the UI and on the server (403 otherwise). It offers a full server backup (every document incl. deleted, all app data) and a reset that needs "DELETE ALL DATA" typed; users, sessions and the audit log survive a reset.
9	DDS-SRS-009	**Save document to the VS Code project folder**	Done	Export to VS Code saves the document (.md, with .docx) into the Coder workspace's specs/docs folders via the file bridge; falls back to browser download if the workspace is offline.
10	DDS-SRS-010	**Download documents**	Done	Download as .docx (plus .md / JSON); Google Docs export also available.
11	DDS-SRS-011	**Admin audit log of user sessions**	Done	The server records every sign-in/sign-out (user, role, login, logout, expiry) in Redis; admin-only view in Data Management (GET /api/audit/sessions, 403 for others). Covers all users on all machines; not editable from a browser.

Test Manager

DDS-SRS-012 – 032 · 11 done · 6 partial · 4 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
12	DDS-SRS-012	**Hierarchy: Projects > Suites > Sections > Cases**	Done	Projects > Suites > Sections > Cases, all stored on the server. Project picker; sections nest to any depth (parent checked, no loops); each case belongs to one section.
13	DDS-SRS-013	**Test case fields (ID, steps, post-conditions, attachments)**	Done	Sequential TC-001 IDs, title, pre-/post-conditions, type, priority (Critical / High / Medium / Low; old P1-P4 converted), steps with expected results, multiple attachments incl. log files, evidence screenshot.
14	DDS-SRS-014	**Revision history for test cases**	Done	Every change to a test case is a server-side revision (who, when, changed fields; latest 50). History view with a before/after table and restore.
15	DDS-SRS-015	**Import/export test cases as CSV, XML, JSON**	Done	Import and export test cases as CSV, JSON and XML (plus .xlsx / Markdown / .robot export).
16	DDS-SRS-016	**Create test runs from cases or whole suites**	Partial	Correction to the 2 Nov report: only whole-suite runs work. The 'Select specific test cases' and 'Dynamic Filtering' options in Add Test Run are shown but not implemented (every case of the suite is included), there is no per-section choice, and cases in the bin are included too.
17	DDS-SRS-017	**Manual execution statuses**	Done	Per-case status in a run: Passed, Failed, Blocked, Retest, Untested, Skipped.
18	DDS-SRS-018	**Attach Jira/GitHub issue IDs to failed results**	Done	Jira / GitHub issue IDs can be attached to each failed result in a run (Defects tab).
19	DDS-SRS-019	**Register remote test servers (URL, auth, payload)**	Not started	No server registry: the app talks to one backend set at deploy time (VITE_API_BASE_URL). No UI for endpoint URL, auth credentials or payload parameters.
20	DDS-SRS-020	**Trigger remote runs from the UI via REST**	Partial	'Run Test' pushes the suite's Robot script and starts a run via REST - against a mock remote Test PC (randomised results); no real hardware integration yet.
21	DDS-SRS-021	**Backend polling service (interval, states, metrics)**	Partial	Polling runs in the browser every 30 min with RUNNING/COMPLETED states. Needs: backend worker (arq on Redis), configurable 5/10/30 s interval, QUEUED/FAILED states, completion %.
22	DDS-SRS-022	**Real-time UI updates without refresh**	Not started	No push updates (no WebSocket/SSE); results appear on the next poll.

Test Manager (continued)

DDS-SRS-012 – 032 · 11 done · 6 partial · 4 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
23	DDS-SRS-023	**Metrics dashboard (completion %, donut, velocity)**	Partial	Run view has a pass/fail donut chart and pass percentage. No velocity/trend graphs.
24	DDS-SRS-024	**Downloadable summary reports (PDF, HTML, CSV)**	Done	Per-run summary report as HTML, CSV and print-ready PDF (IEEE 829 sections).
25	DDS-SRS-025	**Compare runs to detect regressions**	Done	Compare tab: any run vs an earlier run of the suite - regressions, fixed, still failing, new, removed; pass-rate change. Run cards flag regressions vs the previous run.
26	DDS-SRS-026	**Run tests locally and in CI/CD**	Partial	Robot Framework can be run locally in the Coder sandbox (VS Code container); no CI/CD execution path.
27	DDS-SRS-027	**Jenkins and Bitbucket integration**	Not started	No Jenkins or Bitbucket integration.
28	DDS-SRS-028	**Scheduled runs**	Not started	No scheduled runs.
29	DDS-SRS-029	**Integrated code editor with AI generation**	Done	Coder screen embeds VS Code (code-server) with the Cline AI agent; Test Management generates test cases and Robot scripts from a Technical Analysis document with AI (Gemini/OpenAI/Ollama).
30	DDS-SRS-030	**Connect existing code**	Partial	The Coder workspace can hold existing code (mounted folder / git clone in its terminal); no in-app 'connect repository' option.
31	DDS-SRS-031	**Consolidated manual + automated report**	Done	The run report is consolidated: manual vs automated breakdown, each result's type, and the comparison with an earlier run.
32	DDS-SRS-032	**Share docs/test cases with the AI code editor**	Done	Documents, specs and test suites can be pushed into the AI code editor's workspace (file bridge) for test/script generation.

Release Management

DDS-SRS-033 – 046 · 0 done · 4 partial · 10 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
33	DDS-SRS-033	**Create releases, approve RC>Final, override gates, sign off**	Not started	Release Master is view-only: releases cannot be created, RC->Final approved, gates overridden or signed off in-app.
34	DDS-SRS-034	**Link releases to test runs; log blocking defects**	Not started	Releases link to a Technical Analysis doc and a Test Report doc, not to Test Runs; no defect logging on releases.
35	DDS-SRS-035	**Register build artifacts (checksums, URLs)**	Not started	No build-artifact registration.
36	DDS-SRS-036	**View release status, metrics and release notes**	Partial	Shows release status, story-point burndown, readiness and documentation status. No release notes.
37	DDS-SRS-037	**Create a release with version, type, date, status**	Not started	Cannot create releases. Existing releases (sample data) have version, target date and status (Planning / RC Testing / Approved for GA / Deployed) but no type or customer segment.
38	DDS-SRS-038	**Release phase milestones (freeze ... GA)**	Not started	No release phase milestones (Test Management has test milestones, not release phases).
39	DDS-SRS-039	**Multiple RC builds per release**	Not started	No Release Candidate builds per release.
40	DDS-SRS-040	**Image details per build (tag, SHA, URL, checksum)**	Not started	No image tracking (tag, commit SHA, URL, SHA-256, build ID).
41	DDS-SRS-041	**Promote an RC image to Final**	Not started	No RC -> Final promotion.
42	DDS-SRS-042	**Link test runs to RC images**	Not started	Test Runs cannot be linked to RC images.
43	DDS-SRS-043	**Automatic release-readiness gates**	Partial	A Quality Gate verdict (GO / Pending QA / NO-GO) is shown, but it is a stored value, not evaluated from pass rate, open blockers or mandatory suites.
44	DDS-SRS-044	**Link Jira/GitHub release blockers**	Partial	Each release lists its Jira tickets, but from sample data - the 'Sync JIRA' button is simulated; no blocker flag.
45	DDS-SRS-045	**Multi-role electronic sign-offs**	Partial	Multi-reviewer sign-off exists for documents (Reviewers Log). Releases have a single sign-off owner; no multi-role release sign-off.
46	DDS-SRS-046	**Immutable audit log for release actions**	Not started	No audit log for promotions, status changes, gate overrides or sign-offs.

Non-functional requirements

DDS-SRS-071 – 076 · 2 done · 1 partial · 3 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
71	DDS-SRS-071	**UI navigation under 1.5 s**	Done	Measured on a production build against the real API (e2e/navigation-perf.spec.ts): first visits 27-378 ms, later visits 27-64 ms, every screen.
72	DDS-SRS-072	**Polling supports 100 concurrent runs**	Not started	No backend polling service yet (mock runs are in memory); not load-tested.
73	DDS-SRS-073	**5,000+ case suites load in under 500 ms**	Done	A 5,000-case suite opens in ~200-360 ms end to end (server ~64 ms). Cases are stored per suite in Redis and fetched by key - the equivalent of an indexed lookup, no table scan; the list shows 100 per section. Checked by e2e/test-management-perf.spec.ts.
74	DDS-SRS-074	**HTTPS / TLS 1.3 in transit**	Not started	Served over plain HTTP; TLS 1.3 is a deployment step (reverse proxy) not yet done.
75	DDS-SRS-075	**Server API keys encrypted at rest (AES-256)**	Not started	No external test server keys stored yet (see 019). Server-side AI keys live in environment variables, not encrypted at rest.
76	DDS-SRS-076	**JWT and OAuth2/SSO authentication**	Partial	Real login with server-side sessions (httpOnly cookie, bcrypt, rate-limited), every sign-in audited - deliberately not JWT so logout revokes instantly. No SSO yet.

Decisions needed to proceed

#	DECISION	WHY IT IS NEEDED
1	**Background job queue**	Decided: arq (async, fits FastAPI, and uses the Redis the app now runs). Needed for the polling worker (021, 072).
2	**Test PC / hardware API**	Not available yet. Phase 2 will be built against a test-server API we define (the mock Test PC implements it); the real Test PC follows it or gets an adapter (019-021).
3	**Jira, Jenkins, Bitbucket access**	Still needed: instances and service credentials for 027 and real blocker sync (044). Release blockers are entered by hand meanwhile; 018 is done with manually entered issue IDs.
4	**SSO provider**	Which identity provider for 076 (e.g. Azure AD / Okta / Google Workspace).
5	**JWT vs. sessions**	076 asks for JWT. Login currently uses server-side sessions on purpose: logout and revocation take effect instantly. Confirm whether JWT is a hard requirement.
6	**Priority scale**	Decided: renamed to Low / Medium / High / Critical; old P1-P4 values are converted automatically (013 done).
7	**Deployment target**	Still needed for TLS 1.3 (074). The performance targets (071, 073) are met and measured without it.